Teaching KG management with SPARQL Anything

Pedagogical experience: three projects, three cohorts

Ryan Shaw, University of North Carolina at Chapel Hill

Façade-X tutorial, ISWC 2026, Bari

Who does the work

Students in information and library science

- Undergraduates and Master's students
  Future catalogers, archivists, curators, data librarians. Not programmers.
- They know their domain's records
  A MARC 300 field, a gazetteer's “see” reference, a ware's date range.
- They learn the Semantic Web standards in one course
  RDF, RDFS, OWL, SPARQL, SHACL. No programming language.

So the tools have to be the standards they've just learned.

INLS 620

The prerequisite: Distributed Knowledge Graphs

	topics	assignment
Aug	Git, GitHub, VS Code, forms of data	Hello world
Sep	Semantic data modeling, the Web, RDF	Data modeling · The Web
Oct	RDFS and RDF vocabularies, Wikidata	RDFS
Oct–Nov	SPARQL, shape constraints (SHACL)	Wikidata and SPARQL
Nov	OWL, semantic data modeling in practice	SHACL and OWL

aeshin.org/teaching/inls-620

Three projects

Each a class or research team, each with a partner

	years	sources	transformation
NC Gazetteer North Carolina places and their history	2020–23	CSV	tarql
Material Fall of the Roman Empire A historian's archaeological data on Late Antique Iberia	2025–26	CSV, XLSX, GeoJSON	SPARQL Anything
Artists' Books in UNC's Sloane Art Library and the literature citing them	2026–	Zotero, CSV, MARCXML	SPARQL Anything

Project 1: tarql

NC Gazetteer: one CONSTRUCT query per contribution

```
CONSTRUCT {
  ?PLACE a nct:School ;
    skos:prefLabel ?preflabel ;
    ncv:county ?COUNTY ;
    ncv:budgetYear ?budget_year ;
    ncv:rosenwaldType ?rosenwald_type .
}
WHERE {
  BIND (tarql:expandPrefixedName(
          CONCAT('ncp:', ?id)) AS ?PLACE)
  BIND (tarql:expandPrefixedName(
          CONCAT('ncp:', ?county)) AS ?COUNTY)
}
```

- The template is Turtle they can already read.
- Each CSV column is a variable.
- The questions are about the domain.

  Is a Rosenwald school a school or a place? Is a budget year a date?

`contributions/2022-05-05-rosenwald-schools-ADD`: 308 schools, three students.

But only CSV

So everything else became framework

```
# GeoNames JSON, pasted into a CSV cell
BIND (f:jsonPointer(?geonames_json, '/lng') AS ?_LNG)
BIND (IF(BOUND(?_LNG), ?_LNG,
  f:jsonPointer(?geonames_json, '/geonames/0/lng'))
  AS ?LNG)
```

- JSON from an API went into a spreadsheet cell, and out through a SPARQL function written in Java.
- A query over another query's output needed a Bash script to chain them.
- Versions, diffs and JSON-LD framing: two more Java programs.

About 740 lines of Java, Bash and Gradle around the queries.
Students could write the queries; only I could maintain the rest.

Project 2: SPARQL Anything

Fall of Rome: one query, two formats

```
CONSTRUCT {
  ?place a pno:Place ; rdfs:label ?name ;
    skos:closeMatch ?wikidata ;
    geo:hasGeometry [ geo:asWKT ?wkt ] .
}
WHERE {
  SERVICE <x-sparql-anything:location=municipalities.csv> {
    ?row xyz:Municipality ?name ;
         xyz:municipality_wiki_uri ?wikidata .
    BIND (fx:entity(for:, "place/",
            REPLACE(?name, "[^a-zA-Z0-9]", "_")) AS ?place)
  }
  SERVICE <x-sparql-anything:location=municipalities.json> {
    ?f xyz:properties/xyz:shapeName ?name ;
       xyz:geometry ?wktString .
    BIND (STRDT(?wktString, geo:wktLiteral) AS ?wkt)
  }
}
```

- The join is a shared variable.
- No JSON in a cell, no custom function, no chaining script.
- Two students wrote this one.

  Spanish and Portuguese municipalities, for a gazetteer of Iberia.

The pipeline is the syllabus

Fall of Rome: every stage is a standard from INLS 620

stage	how	in INLS 620
transform	SPARQL CONSTRUCT, over CSV and GeoJSON	SPARQL
check	SPARQL SELECT: count what came out	SPARQL
validate	SHACL shapes, one per source	shape constraints
infer	RDFS and OWL	RDFS, OWL
publish	SPARQL SELECT + HTML templates (Snowman)	SPARQL

Each of six students took a source the whole way: its query, its shape, its pages.

Project 3: library data

Artists' Books: MARC is just another façade

```
SERVICE <x-sparql-anything:location=marc.zip> {
  fx:properties fx:archive.matches ".*\\.xml" .
  ?archive fx:anySlot ?file .
  SERVICE <x-sparql-anything:> {
    fx:properties fx:location ?file ;
                  fx:from-archive "marc.zip" .
    ?record fx:anySlot ?df .
    ?df xyz:tag "300" ; fx:anySlot ?sf .
    ?sf xyz:code "a" ; rdf:_1 ?extent .
  }
}
```

- About 7,900 catalog records, one MARCXML file each, read straight from a zip.
  300 $a is the extent: “64 unnumbered pages.”
- Students read MARC on sight: that's library school.
  One added languages from 008 and 041; another curated the construction vocabulary against two artists'-book glossaries.

What's still code

Procedural code moved to the edges

	still procedural	because
NC Gazetteer	JSON, chaining, versioning	tarql reads only CSV
Fall of Rome	GeoJSON → WKT (15 lines of Python)	a geometry conversion
Artists' Books	harvesting catalogs; fuzzy deduplication Z39.50, SRU, a museum API	getting data, and judgment calls The judgments are CSV files students edit.

Transformation is declarative. Acquisition and matching aren't, yet.

What we learned

1. One semester of learning technological standards is enough.

   Students arrive knowing SPARQL. Façade-X means there's no second language to learn.

2. Formats, not functions, remove the framework.

   tarql made CSV declarative. Everything else needed code until every format had a façade.

3. Students can spend their time on the domain.

   Rosenwald schools, African Red Slip Ware, MARC 300: not parsers, loops or build systems.

`github.com/ncgaz/ncg-dataset` `github.com/dkglab/fall-of-rome` `github.com/dkglab/artists-books`